Architecture and Implementation of an FPGA-Based CNN Accelerator Using Hardware–Software Co-Design

Name: TEJO PRIYA BODDU
ID No: N200429
Guide: Mr. Shivlal Kethavath
ECE, RGUKT-NUZVID

Abstract

This project presents the design and implementation of a custom Convolutional Neural Network (CNN) hardware accelerator using Vitis High-Level Synthesis (HLS) on FPGA. The objective is to reduce computation latency and power consumption compared to software-based CNN execution. The accelerator is designed using C/C++ and converted into RTL using HLS. It is integrated into a complete system using MicroBlaze, AXI interconnect, and Block RAM (BRAM). The design is implemented and verified through synthesis, placement, routing, and timing analysis.

Introduction

- CNNs are widely used in image and signal processing
- Software execution of CNNs is slow and power-intensive
- FPGA provides parallelism and reconfigurability
- Hardware acceleration improves speed and energy efficiency

Problem Statement

- High computation latency in CPU/GPU-based CNNs
- High power consumption in embedded systems
- Need for real-time, low-power AI hardware
- Lack of optimized hardware accelerators for edge devices

Literature Review

- CNN hardware acceleration research focuses on higher computation speed and lower energy consumption.
- FPGA-based accelerators often outperform CPU/GPU in performance-per-watt.
- High-Level Synthesis (HLS) tools, e.g., Vitis HLS, simplify hardware design using C/C++.
- HLS-based FPGA design methodologies motivated the approach used in this project.

Literature Review

Key Observations from Prior Work:

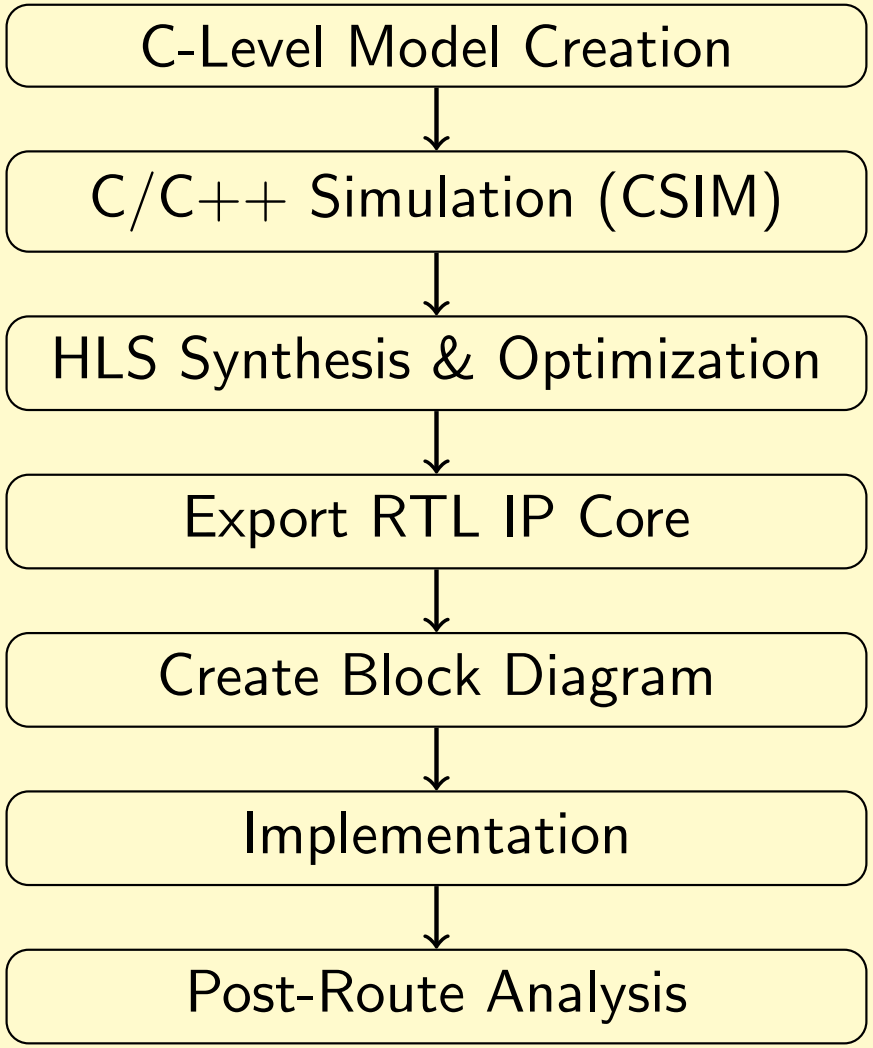
- Early FPGA CNN accelerators used manual RTL, increasing design complexity
- HLS-based approaches reduce development time while maintaining high performance
- AXI-based architectures enable integration with MicroBlaze processors
- BRAM memory provides fast on-chip storage for feature maps and weights
- Loop unrolling, pipelining, and memory partitioning are essential for high throughput

These insights guided the system architecture and optimization strategy in this project.

Objectives

- Design a CNN convolution accelerator in hardware
- Use Vitis HLS for hardware IP generation
- Integrate accelerator with MicroBlaze processor
- Analyze timing, power, and resource utilization

Work Flow


C-Level Model Creation
C/C++ Simulation (CSIM)
HLS Synthesis & Optimization
Export RTL IP Core
Create Block Diagram
Implementation
Post-Route Analysis

CNN Accelerator Design (Vitis HLS)

The convolution operation is implemented using C++ in Vitis HLS. Performance is improved using loop pipelining, loop unrolling, and array partitioning. Fixed-point arithmetic reduces hardware complexity and power.

Design Stage	**Latency (ns)**	**Iter. Latency (cycles)**	**II**	**DSP**
Initial (No Pragmas)	2510	252	252	9
Partially Optimized	820	83	72	54
Fully Optimized	340	35	18	54

Table: HLS Optimization Results Summary

AXI Interface Architecture

- AXI4 for high-speed memory access
- AXI4-Lite for control and status registers
- AXI Interconnect routes all transactions

MicroBlaze Soft Processor

MicroBlaze is a 32-bit soft processor implemented using FPGA logic. It controls the accelerator via AXI-Lite and manages memory transfers. Clock is generated by Clock Wizard.

BRAM and AXI BRAM Controller

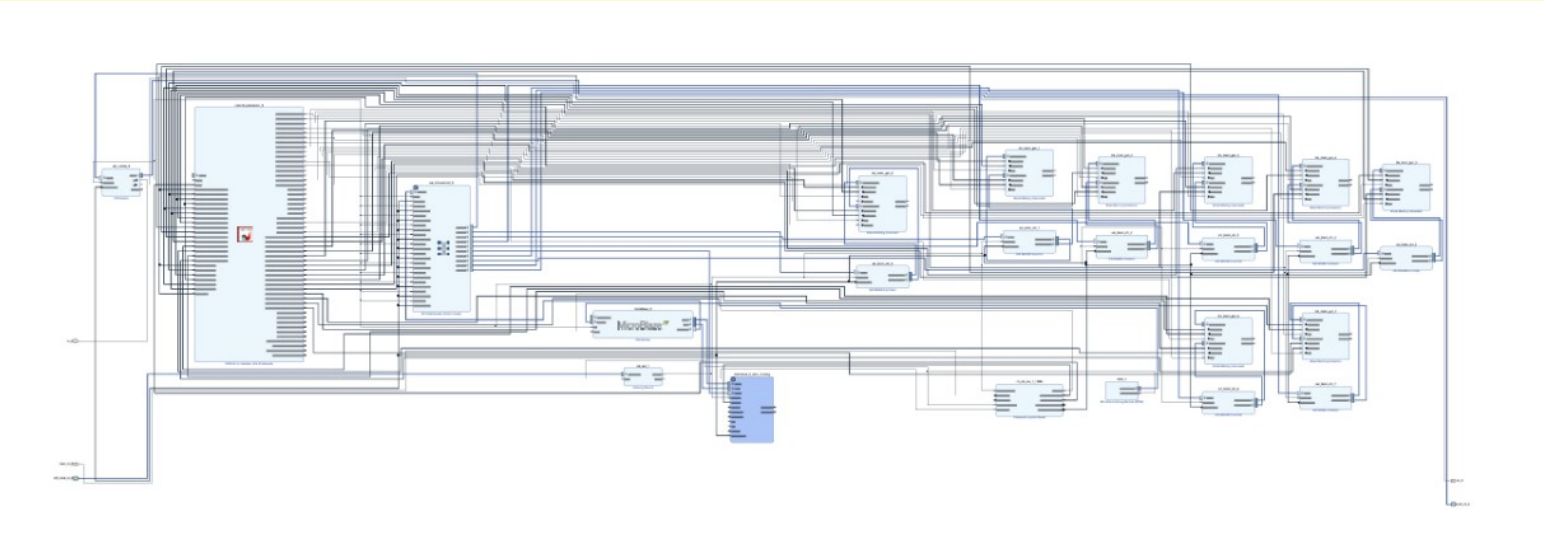
Block RAM stores inputs, weights, and outputs. AXI BRAM Controller enables fast processor-to-memory communication.

Custom CNN Accelerator IP

The CNN accelerator is exported from Vitis HLS as a reusable RTL IP and added to the Vivado IP catalog.

CNN System Block Design

The system consists of a MicroBlaze soft processor connected to a custom CNN accelerator IP through AXI Interconnect. BRAM memories store feature maps and weights. AXI provides communication between processor, memory, and accelerator.



Implementation Process

- RTL Synthesis
- Floorplanning
- Placement
- Routing

Utilization Reports

Logic and memory blocks were optimally distributed.

DATAFLOW DESIGN - xc7a100tcsg324-3 make active

Tcl Console | Messages | Log | Reports | Design Runs | Dataflow Hierarchy Browser

Name	Blackboxes	IO	Block RAM	Register	LUT	LUT RAM
dnn...per	0	4	2	2390	3544	147
dnn_...em)	0	1	2	2390	3544	147
micr...0_0)	0	0	0	909	1062	106
axi_...0_0)	0	0	0	109	102	18
mdm..._0)	0	0	0	110	105	7
axi_...0_0)	0	0	0	440	627	13
axi_...0_0)	0	0	0	69	167	0
axi_...7_0)	0	0	0	69	167	0
axi_...6_0)	0	0	0	69	167	0
axi_...5_0)	0	0	0	69	167	0
axi_...4_0)	0	0	0	69	167	0
axi_...3_0)	0	0	0	69	167	0
axi_...2_0)	0	0	0	69	167	0
axi_...1_0)	0	0	0	69	167	0
mic...XG)	0	0	2	232	289	2

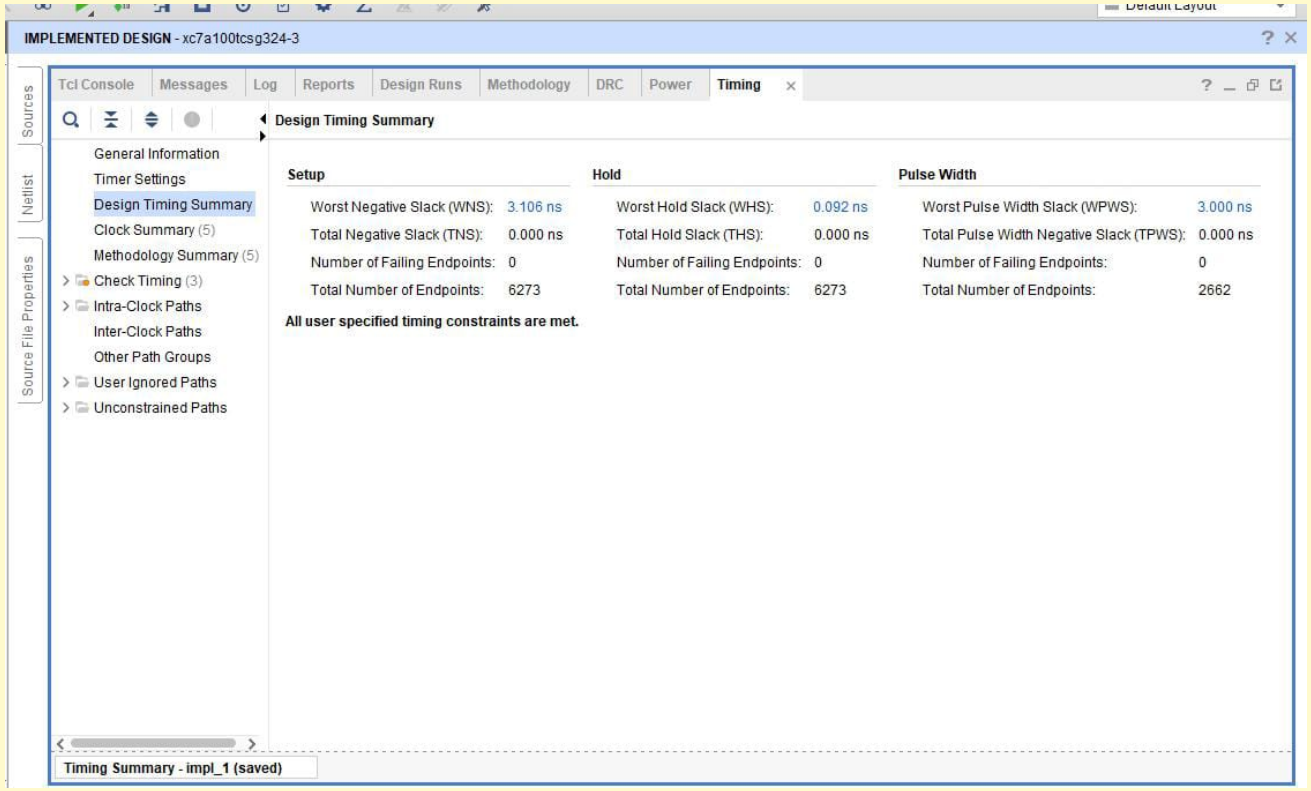
Routing Results

All connections were successfully routed with minimal congestion.

```
Phase 8 Route finalize
Router Utilization Summary Global Vertical Routing
Utilization = 0.569787 % Global Horizontal Routing
Utilization = 0.848323 % Routable Net Status*
*Does not include unroutable nets such as driverless and
loadless.
Run report_route_status for detailed report.
Number of Failed Nets = 0
(Failed Nets is the sum of unrouted and partially routed
nets)
Number of Unrouted Nets = 0
Number of Partially Routed Nets = 0
Number of Node Overlaps = 0
Phase 8 Route finalize | Checksum:  29695f10c Phase
9 Verifying routed nets Verification completed
successfully
```

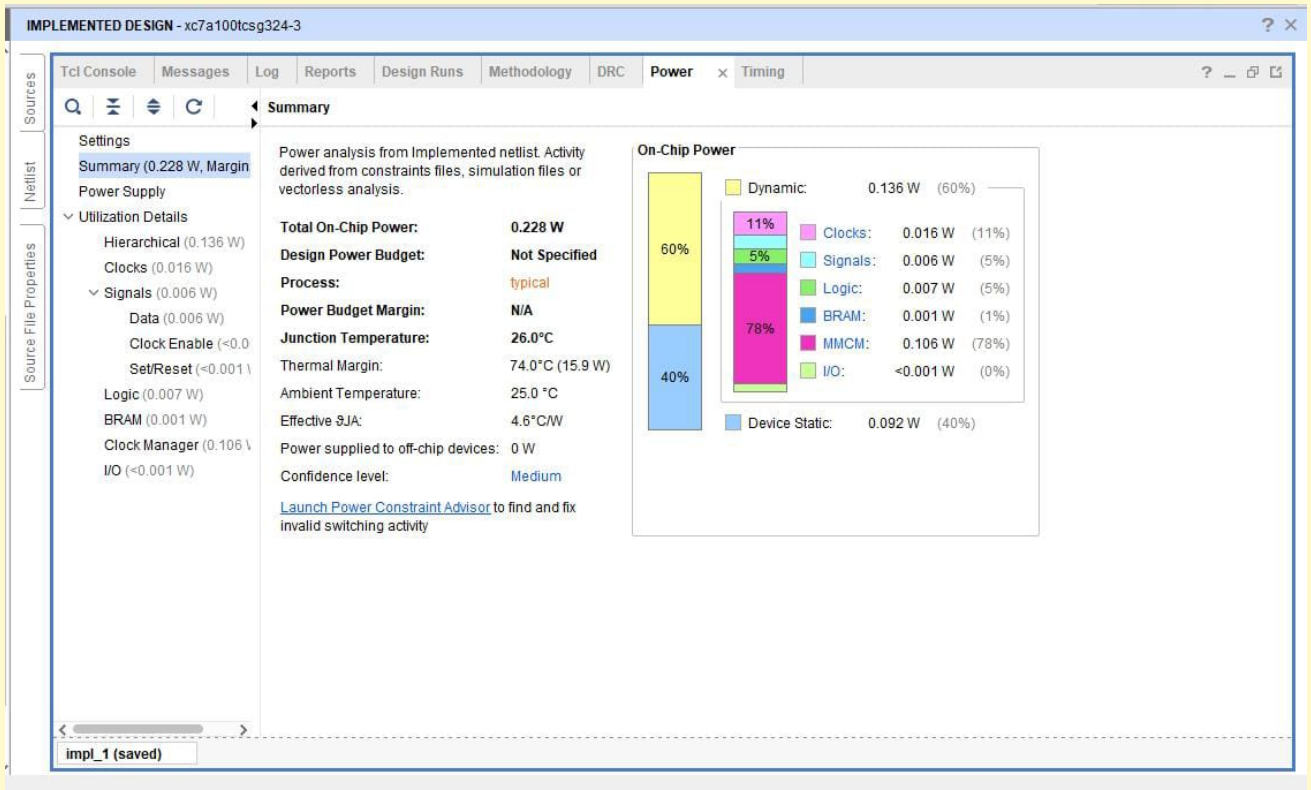
Timing Results

The implemented design meets all timing constraints. The Worst Negative Slack (WNS) achieved is 3.106 ns, with Total Negative Slack (TNS) = 0.000 ns, indicating no setup violations. The Worst Hold Slack (WHS) is 0.092 ns with zero failing endpoints.



Power Results

The total on-chip power consumption of the design is 0.228 W. The dynamic power is 0.136 W (60%), while static power is 0.092 W (40%).The design operates at a safe junction temperature of 26°C.



Conclusion

The CNN accelerator was successfully implemented using Vitis HLS and Vivado with improved performance and reduced latency.

Future Work

- Deeper CNN layers
- DDR integration
- Pooling and activation in hardware
- Real-time inference

Thank You

Tejo Priya Boddu
ECE, RGUKT - NUZVID